BUrl Technical Report

by
Benjamin Bonafede
Will Choi
Alexa Horvath
Conor McNichols
Thao Nguyen
Brian Scotto

Client: Todd Merriett

CSCI 476 Senior Design

5/2/2025

Table of Contents

Executive Summary

This document serves to outline the process and experience of developing BUrl. BUrl is an online URL shortener exclusively for Bucknellians developed by a Senior Design team of 6 Computer Science students. This service is intended to mimic all the functionality of existing shortening services while being free and giving users analytics related to links. This report will first provide an overview of the problem BUrl was created to solve. The design principles, implementation, and decision making sections will inform how building BUrl was approached and what considerations were paramount. The final sections will provide an overview of the results, a reflection of the development experience, and potential future directions of BUrl.

Problem

Bucknell University's Engineering External Relations & Communications team, led by Manager Todd Merriett, faces challenges in effectively sharing professional-looking URLs while maintaining control and tracking engagement. Currently, Todd relies on third-party URL shorteners like TinyURL and Bitly, which, while functional, lack a polished and institution-branded appearance. Additionally, these external services do not provide integrated analytics, requiring Todd to manually compile data from multiple sources such as email marketing software and social media platforms all to assess link performance. Furthermore, the absence of control means link monitoring cannot be restricted to university-affiliated users and groups when necessary. To address these issues, this project aims to develop a custom URL shortener tailored to Bucknell University's needs. The system will not only enhance the professional presentation of shared links but also provide comprehensive built-in analytics and control features, creating a centralized and efficient solution for managing link distribution and engagement tracking.

Primary Objectives:

1. Achieve the URL shortener functionality of third-party solutions with Bucknell-specific branding.
2. Post the project as a web client available on the Bucknell network for non-technical users to use and view link analytics.
3. Receive a user satisfaction score of 80%

User Interface Design Principles

A key piece of this project was creating a platform that is intuitive and accessible for all Bucknellians. We followed standard design principles to develop a user-friendly interface with a modern and sleek feel. The user stories which guide the design highlight a few major needs including simplistic functionality and shareability.

"*Efficiency is key for my role. I need a straightforward tool to manage and share information without technical hurdles*." -Frederik Jensen (Administrative Staff)

"*I want to easily spin up a website with a custom Bucknell URL so my students can access our class site more easily to sign up for our final project.*" -Nathan Young (Professor)

It is a priority for users to be able to create, share, and access BUrl link's with ease. To target these concerns, features were implemented to maximize usability such as the QR code generator, "Copy Link" button, and AI suggestions. The overall user experience was developed through iterative mockup reviews, streamlined user flow diagrams, user-friendly design templates, and client collaboration.

The team's background and experience supported the user interface (UI) design as well as the properties of the software used. The Phoenix/Elixir framework enhances intuitive design with its seamless compatibility with industry standard Tailwind templates, built-in scalability to address accessibility concerns, and mobile-friendly architecture.

To evaluate the execution of the design principles, both users and online tools were utilized. Following the Alpha Release of BUrl, BUrl's client and several Bucknell students and faculty began using BUrl. Some users had in-person meetings to deliver feedback; others received a brief survey evaluating their experience with BUrl. The survey included open-ended and rating questions. Example questions are shared in *Figure 1*. To support usability, accessibility auditing was done to raise Web Content Accessibility Guideline (*WCAG*) compliance [2]. The audit applied results from WebAim's Wave tool, WCAG's published documentation, and Phoenix's online resources. Feedback and edits were addressed to improve BUrl for Beta Release.

How would you rate the website's intuitiveness? *(Confusing) 1 2 3 4 (Perfectly Intuitive)*
How often did the website feel frustrating to use? *(Rarely) 1 2 3 4 (Frequently)*

Figure 1: Preview of Survey Questions

Software Design Principles

The development of this URL shortener was guided by principles of scalability, maintainability, and performance. The backend was developed with scalability in mind, utilizing efficient routing, caching mechanisms, and optimized database queries to ensure high performance under varying load conditions.

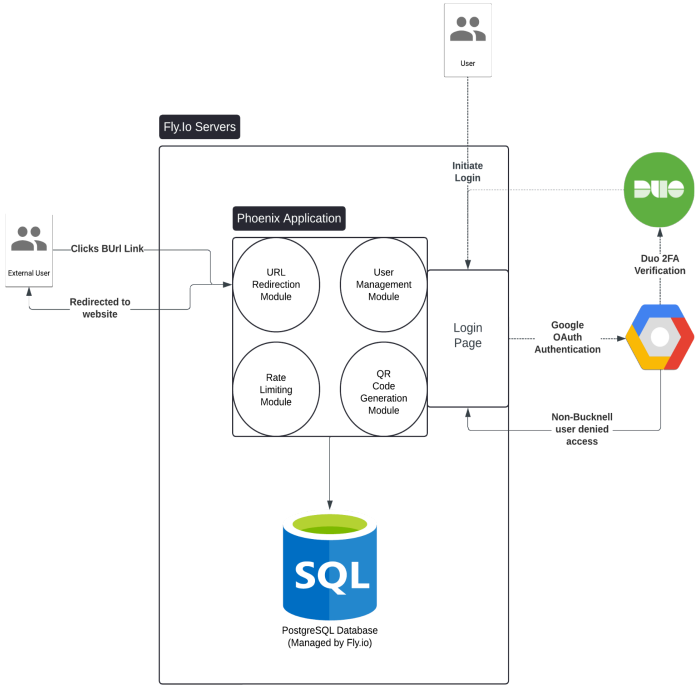

Figure 2: UML Diagram of our Software Design

As seen in the diagram above, our program is deployed via fly.io enabling seamless horizontal scaling and global edge deployment to minimize latency for distributed users. The codebase is structured to support continuous integration and deployment (CI/CD), allowing for rapid iteration and feature expansion; This is a core aspect of our system design which we took advantage of throughout the development process by seamlessly integrating new features at the request of the client. Additionally, the system was designed with long-term maintainability in mind, featuring well-documented APIs and logical code organization that facilitates onboarding and collaboration.

This approach results in a robust, professional-grade URL shortener capable of growing with the needs of Bucknell University and its employees.

Implementation

The project primarily consists of a backend service developed using the Elixir programming language (version ~1.17.3) and the Phoenix web framework (version ~1.7.14). The backend manages data storage with PostgreSQL, utilizing Ecto, an object-relational mapper (ORM) that simplifies database operations. PostgreSQL with Ecto was specifically chosen due to its native integration with Phoenix and as an opportunity for the team to acquire valuable new database management skills.

Authentication is handled through the Ueberauth Elixir package, which provides straightforward and secure integration with Google OAuth, managing user identities reliably.

Ueberauth was selected due to its widespread adoption and robust security practices in the Elixir community, simplifying OAuth interactions significantly compared to manual implementations.

Deployment occurs on Fly.io, a cloud hosting platform recommended by our professor. Fly.io was selected due to its simplicity and compatibility with Phoenix applications. While the initial intention was to deploy on Bucknell's Linux servers, administrative constraints led us to adopt Fly.io.

Front-end development leverages Phoenix LiveView, a choice driven by the desire to explore new skills in real-time web applications and fully utilize Phoenix's native functionality. Styling is provided by TailwindCSS, chosen for its ease of use and utility-first approach that rapidly accelerates UI development. Frontend asset compilation employs Tailwind for CSS and esbuild for JavaScript transpilation, orchestrated by Mix (the Elixir build tool) and Hex (the package manager).

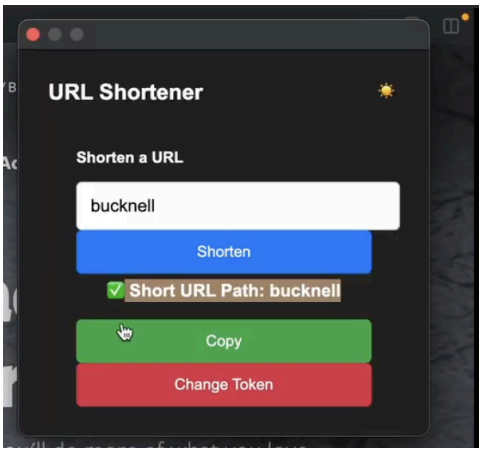

Figure 3: Chrome Extension In Use

To improve the URL shortening process, a custom Chrome Extension (Fig. 3) was developed using JavaScript, HTML, and CSS compliant with Manifest Version 3. Manifest V3 was selected to meet current Chrome Web Store standards and leverage enhanced security and performance. The extension uses secure browser permissions (`activeTab`, `storage`) to interact with the backend securely via clearly defined local (`*localhost*`) and production domains (`*bucknell.fly.dev*` and `*burl.live*`).

The comprehensive environment setup process is streamlined through a detailed shell script (`setup.sh`). This script installs critical development tools, including ASDF for runtime management, Erlang and Elixir, Hex package manager, Node.js for assets, and PostgreSQL. The script ensures proper installation of dependencies, sets up the PostgreSQL database, generates the `SECRET_KEY_BASE` for Phoenix security, and guides the user through initial project creation and dependency installation. Although API keys are not managed through an `.env` file, the script instructs users to store keys in their shell configuration files (`.zshrc`, `.bashrc`) or as Fly.io secrets for secure deployment and retrieval at runtime.

The backend employs several libraries carefully chosen for functionality and efficiency. The `qr_code` library was selected for QR code generation due to its lightweight nature, allowing browser-side generation with custom branding features (logo and colors). Alternative libraries like `eqrcode` were considered but did not meet the customization and performance needs.

```
    def get_geo_location(ip_address) do
      # Check for localhost or development IPs
      if is_development_ip?(ip_address) do
        # Return a development location
        get_development_location()
      else
        result = Geolix.lookup(ip_address, where: :city_database)

        case result do
          %Geolix.Adapter.MMDB2.Result.City{
            city: %{names: %{en: city}} = _city,
            country: %{names: %{en: country}} = _country
          }
          when not is_nil(city) ->
            "#{city}, #{country}"

          %Geolix.Adapter.MMDB2.Result.City{
            country: %{names: %{en: country}} = _country
          }
          when not is_nil(country) ->
            country

          _ ->
            "Unknown"
        end
      end
    end
```

Geolocation Code Block

An Example of our geolocation tracking as implemented above, uses Geolix and the MaxMind GeoLite2 database, which provides a self-contained, locally managed solution without external API dependencies. This choice simplified deployment and ensured reliable, rapid IP-to-location lookups.

Leaflet was adopted for geolocation visualization due to its open-source nature, ease of integration, and independence from external services like Google Maps. It also facilitated the use of custom markers and interactive map components.

Initially, chart generation attempted through native Phoenix LiveView and Tailwind was cumbersome and visually unsatisfying, leading to the adoption of ApexCharts.js. ApexCharts significantly streamlined graph creation, offered superior aesthetics, and reduced development time.

Security and code quality are integral to the project. Static analysis is managed with Credo, enforcing coding standards and best practices, while Sobelow provides real-time security vulnerability scanning for Phoenix-specific issues. Rate limiting and API usage caps are strategically implemented to prevent misuse and mitigate potential financial risks associated with third-party services. The motivation behind stringent security measures stems from the application's intended audience, comprising faculty, alumni, and other distinguished Bucknell community members, where security breaches would significantly impact institutional integrity.

A robust GitLab CI/CD pipeline enhances project reliability, security, and maintainability. The pipeline encompasses several critical stages:

- **Sobelow Scan**
  - Performs automated security scans and generates vulnerability reports.
- **Smoke Tests**
  - Runs targeted integration tests to ensure critical functionalities are operational post-deployment.
- **Slack Notifications**
  - Automatically communicates pipeline results to the development team via Slack, improving visibility and responsiveness to potential issues.
- **Deployment**
  - Automates deployment to Fly.io using Fly CLI, with Docker hosting a GitLab runner on a team member's machine to facilitate these continuous integration processes. GitLab CI was specifically chosen due to institutional availability and ease of integration with Bucknell’s infrastructure.

Performance optimization was thoughtfully considered throughout development. URL redirection operations were implemented asynchronously to minimize user wait times, ensuring immediate response while analytics and click tracking processes are handled separately. Frontend optimization was further achieved by minimizing websocket payloads in LiveView, storing minimal data (primarily IDs) client-side, and fetching detailed information from the backend as needed, thereby significantly enhancing user interface responsiveness and resource efficiency.

Decision Making

One early decision the team had to make was selecting our hosting environment. They initially chose Fly.io because it was easy to use, scalable, and provides support for Phoenix/Elixir applications. The team also considered migrating to Bucknell’s Linux server to cut costs. However, the team had a discussion with a Bucknell’s L&IT staff member where he was honest about the difficulties of maintaining it long term because of their unfamiliarity with the language and framework. After consulting with our advisor, he clarified that our Fly.io set up could be optimized to cost $6 per month by removing a redundant instance. In the end, the team prioritized ease of deployment and sustainability in which they decided to continue with Fly.io.

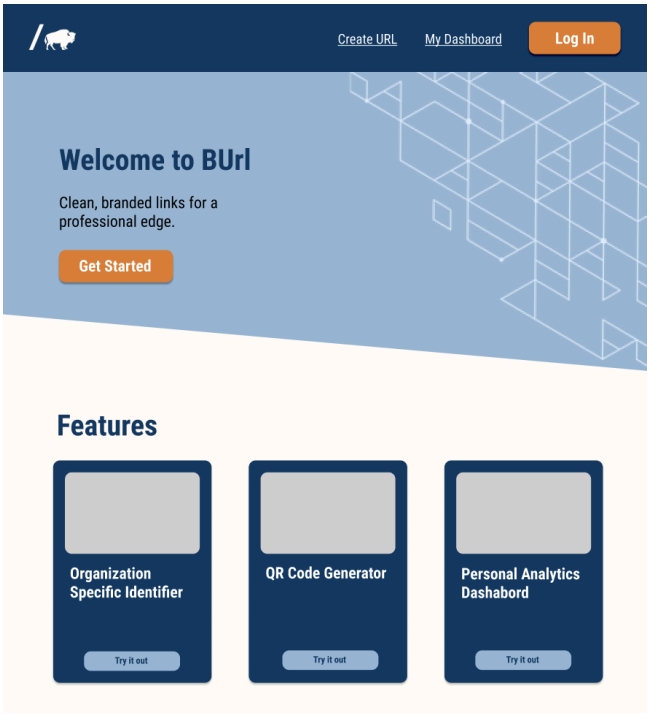

Figure 4: Initial Landing Page from our Figma Mockup

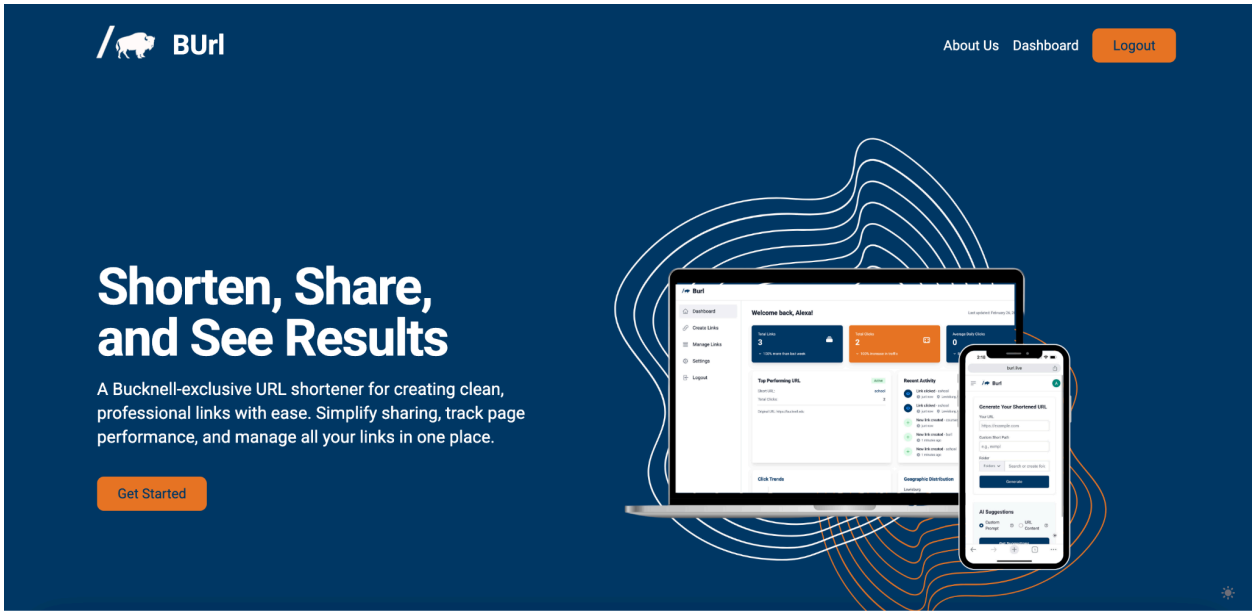

Figure 5: Current Landing Page

Another important decision revolved around the UI. Shown in the images above, the team has redesigned our landing page based on early user challenges. The initial landing page (*Fig. 4*) presented three separate boxes for link management, QR code generator, and analytics dashboard. While it presented our core functionalities, this layout made navigation confusing where once the user selected a feature, there was no intuitive way to return or go to a different feature. This experience did not align with our goal of making BUrl user-friendly. So, the team

redesigned the landing page to be more cohesive and professional, redirecting the user to the dashboard once logged in (*Fig. 5*). Furthermore, they implemented a left-hand menu to allow seamless navigation between all core features. This UI change has significantly improved usability and allows future features to be easily integrated.

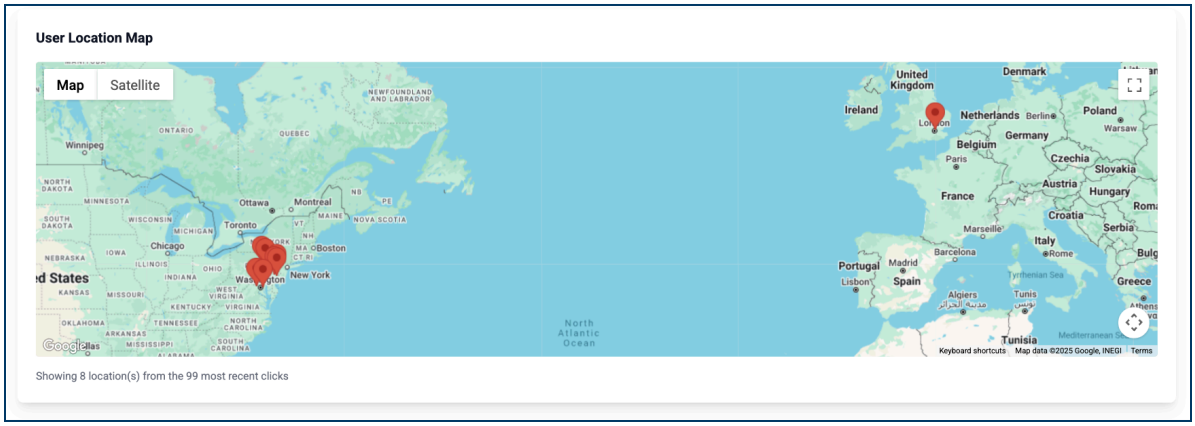

Figure 6: Map Feature

Beyond the UI layout, we also made the decision to include a visual map feature on the analytics page (*Fig. 6*). The team felt that by providing a geographic visual of where the shortened URLs were accessed would be appreciated by users that are interested in tracking outreach trends and provide feedback to them. While this was not in the team's original feature plan, they believed that the map feature added a nice touch to the analytics page and made it easier for the user to understand where their shortened links were being seen without the stress of interpreting raw data tables.

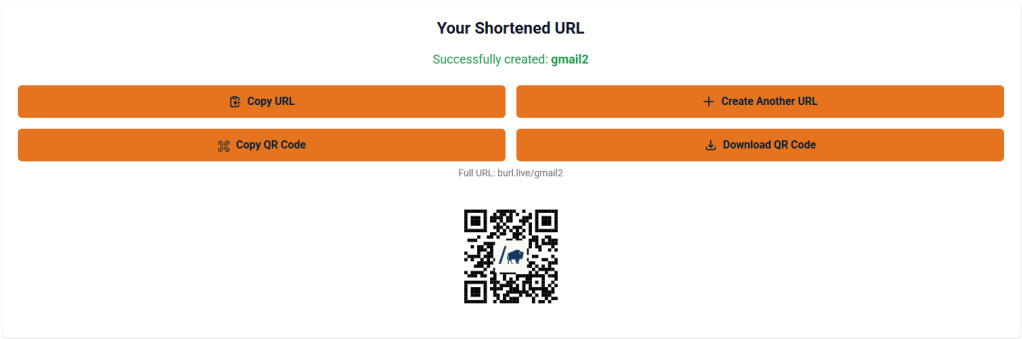

Figure 7: Successful Link Creation Page

The team also focused on making the link creation process as efficient as possible by adding some small but impactful functions. After the user generates a new URL, they are redirected to a confirmation page that also displays four clean action buttons: copy URL, create new URL, copy QR code, and download QR code (*Fig. 7*). This layout makes it accessible for the user while providing a sleek look. The copy URL button allows the user to quickly copy their shortened link without extra clicks, while the create new URL button allows them to start a new

creation without navigating backwards. The QR code options makes sharing easier, offering both a downloadable SVG file or a quick copy function for putting it in materials like flyers. Furthermore, the team thought to have the QR code be stored in the Manage Links table to allow the user to revisit and reuse it at any time. To further enhance convenience, they also decided to develop a Google Chrome extension that would allow the user to create shortened URLs directly from their browser without navigating to the BUrl site. All of these decisions were aimed at improving user experience and engaging the user to come back for more.

Significant Outcomes

BUrl successfully addresses the client's problem by providing a centralized, branded, and user-friendly solution for creating and managing shortened URLs. The platform aligns with the initial goals outlined in the original proposal and demonstrates success through testing results, user feedback, and performance metrics.

Initial Goals:

- Develop a centralized solution for managing link distribution and engagement tracking.
- Launch an exclusive Bucknell University branded URL shortener using a custom domain.
- Deliver an intuitive user interface with greater than 80% user satisfaction.
- Exceed the features and usability of existing URL shortening platforms.

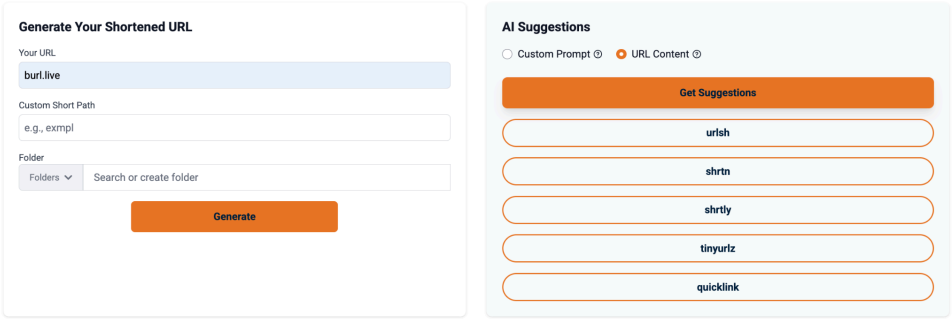

Figure 8: BUrl's Create Link page

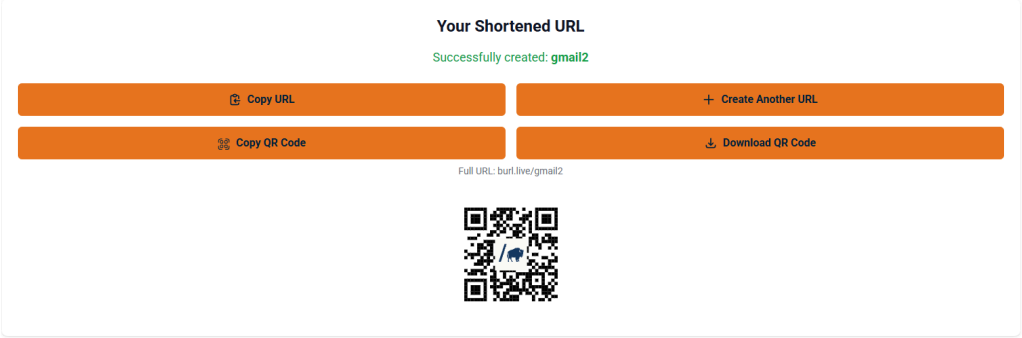

Figure 9: BUrl's QR code generation

BUrl has achieved these goals by implementing a custom domain that ensures all shortened links are professional and recognizable as part of the Bucknell brand. Users can create custom short paths and unique QR codes, enabling them to tailor URLs for specific events, projects, or audiences (*Fig. 8 and 9*).

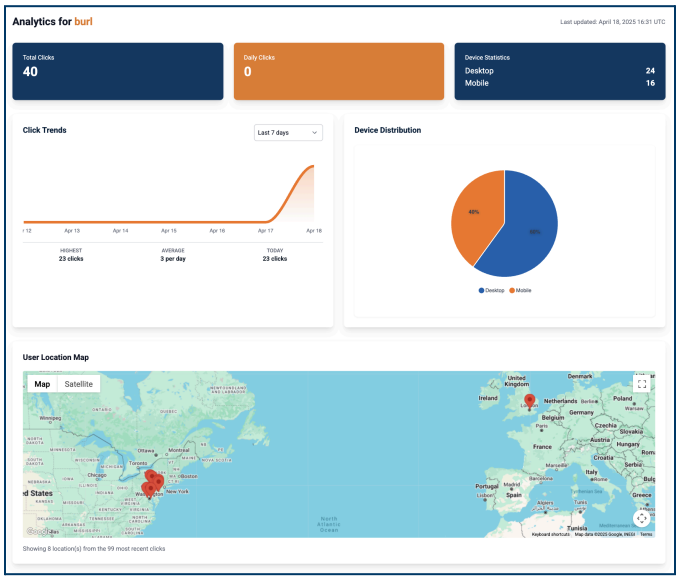

Figure 10: BUrl's Link Analytics page

BUrl includes complete analytics capabilities, giving users insight into their link performance. Key metrics such as total clicks, unique clicks, and device usage are visualized on a user-friendly dashboard (*Fig. 10*). For example, during testing, BUrl successfully recorded accurate click data, including daily activity trends and device types.

Security was another critical focus. BUrl incorporates Google Authentication and admin-based access control to ensure only authorized Bucknell users can access the system. Admins have enhanced privileges, including managing user permissions and viewing site-wide analytics.

Final Features:

- Link Analytics and Data Visualizations
- Google Authentication
- AI-Generated Link Suggestions
- QR Code Generator
- Free Chrome Extension

SCRUM Agile Framework

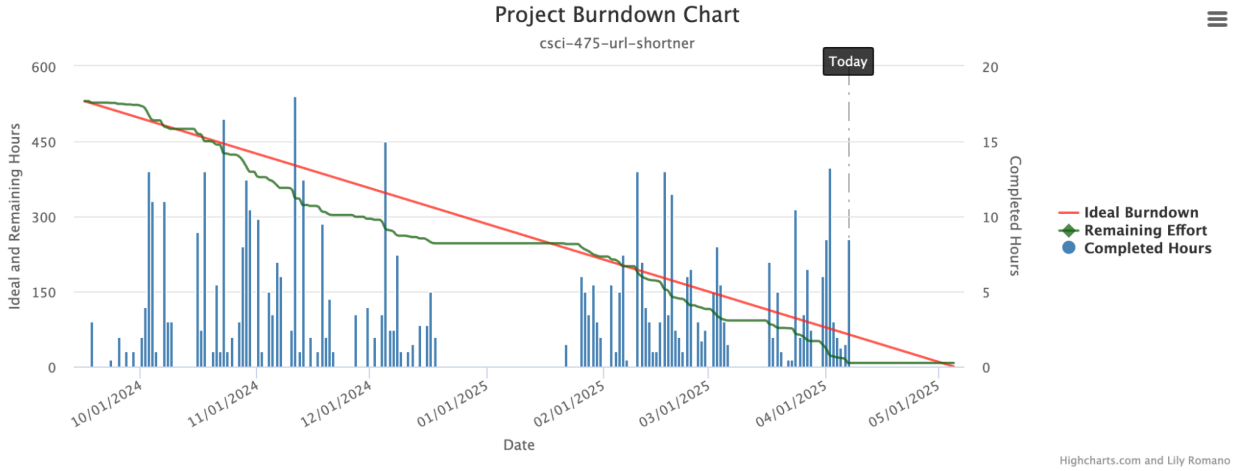

Figure 11: AIEcode Burndown Chart of Sprints 1-8

The project lifecycle of BUrl included documentation, development, and various meetings. To successfully produce sprint deliverables and manage this array of tasks, the SCRUM agile framework was adopted and recorded through AIEcode. The project's final burndown chart is shown in *Figure 10*. The chart shows a nearly ideal burndown trend with a fairly even distribution of work among the six team members.

The BUrl team was able to adhere to the SCRUM agile framework as well as leverage it to create a successful project. A key element of the framework is iterative sprints allowing for shorter term goals to be made and met. During sprint reviews, the BUrl team committed to the prompts and approached it with genuine reflection. The team showed a strong effort in considering areas of improvement and team strengths within the sprint. During the following sprints, the team collectively aimed to improve. One area of improvement noted was the team's need for better and more frequent communication. In the early stages of BUrl, nearly all team members were working on a different feature or page making consistent communication critical. Sticking to the framework's call for frequent scrum meetings while utilizing platforms such as iMessage and Slack led to improved communication.

In adopting the SCRUM agile framework, some challenges were faced. For instance, regularly submitting spent time for tasks throughout the sprints rather than before the sprint review was a challenge at first. During the sprint, this made it difficult to see progress and ensure even workloads were being distributed. One contributor to this was the design of AIEcode. At times, the system felt burdensome to use due to its slow and somewhat incomplete functionality. For instance, to save edits and time spent on a task, the window would need to be saved multiple times and saving items would take a few seconds each. Better comfortability with AIEcode and more team communication helped to rectify this issue. Another struggle was fully completing tasks and certain deliverables by the end of the sprint. This stemmed from underestimating the time required for certain tasks due to unexpected bugs and the added challenge of being new to some technical elements. The BUrl team adapted well to such challenges and used daily sprints and sprint reviews to regroup to ensure all goals were ultimately met and everyone was productive and supported in their work.

Overall, the SCRUM agile framework made BUrl an efficient and manageable project. An evident benefit of the daily SCRUM meetings was the opportunity to check-in with each team member. It prompted team alignment by giving the opportunity to get helped when stuck, deliver feedback, and assign new tasks to maintain progress. Additionally, the documentation from using these methods is valuable for reviewing team progress and for future teams that may work with BUrl and need to understand its development.

Discussion & Reflection

Having now deployed BUrl and made it available for Bucknell students and faculty, the project can be largely deemed as a success. In the original project proposal, the team's goal was to make a URL shortener which matched the functionality of other larger services, to include new features such as QR code generation, and an analytics dashboard, and to have an overall user satisfaction score of 80% or above. From a brief overview of BUrl's current features, it has met and surpassed ours and the client's initial specifications for the project. Surveying our app against eight test users, there was an average score of 94% 4/4 scores for intuitiveness. Comments from the last review meeting before the beta launch with Todd and our users were overwhelmingly positive, with specific critique being mainly related to certain aspects of user flow rather than the core functionality of the site.

Having now nearly concluded BUrl, areas for improvement can largely be identified in parts of its development process. Throughout development, the BUrl team was strong at identifying issues causing inefficiencies miring a previous sprint during a sprint review, and then getting the team to change their habits so that it wasn't an issue in future sprints. One issue that came up relatively consistently though was that tasks would often be carried between sprints instead of being started and ended in one sprint. This was largely due to the team having to get comfortable communicating over with each outside of in-person meetings and getting used to AIEcode. Especially early on in the project, we weren't sure what our individual capacities for work were and how long it took to learn the nuances of technologies like Phoenix and Postgres. As time went on though, we got a much better feel for how long it took for us to look and how to optimally break up tasks among us based upon experience and speed. Over the duration of the project, we also got a feel for how often we should communicate with each and through which channels we should. In the beginning of the project we straddled the usage of multiple communication platforms like Slack, Discord, and text, but by the end we realized that we could most effectively communicate with each other by just using text.

Additionally, the team could have been more mindful about directly tackling the project's main functionality and objectives earlier on in the development. While our stated goals in early sprints was to build out some of the basic redirect functionality and our analytics, we spent most of our time designing our web client's front-end. Although we knew the value of getting our core features completed from our advisor meetings with Professor Fuchsberger and our talks with a UX specialist from Penn State, we created tasks that did not align with our knowledge. Later on in the project the team made sure to directly craft individual tasks and sprint goals based off of initial project objectives. For example, in a meeting with Professor Marchiori in the second semester where we discussed how we didn't have the QR feature yet, we dropped all our less relevant work to implement that.

Finally, by the end of semester we got better at pulling out meaningful feedback during client meetings. During the start of the semester when we had our client meetings, we would typically get very positive feedback from our client without much feedback to better our project. Close to the midpoint of our project, we got much better at asking more pointed questions in order to get a better understanding of what would benefit BUrl. For example, instead of simply asking if our client liked the state of BUrl, we would ask if the service would benefit from the "X" feature or could be improved by changing some specific thing in the UX.

Future Directions

Additional ideas on what data to capture should in the future should also be reflective of user feedback. Currently, BUrl will capture information such as hits, location, and device usage on links. In the future however, users may find it necessary to have a larger spread of categories to analyze from. Some interesting categories that are not currently being analyzed in our application are user demographic information like age and gender, and information related to a user's machine, like their browser.

Outside of improving existing analytics features, more could be done to make BUrl's generated links more customizable. One such feature could be allowing users to password protect their links. Since most links are created by users to circulate amongst the general Bucknell populace it seemed superfluous to add, but if over time the platform is used to circulate protected links for a specific sub-organization like a club or for protected documents like exams, then this feature could be become useful. An additional feature proposed but never implemented was an settable expiration date for links, which would allow users to control how long their links would be active. Again, this feature was never implemented because it didn't align with the needs of present users who always wanted their links to be active, but in the future if the application is used more greatly for things such as event notification, it may be good to have links only be available for a certain period of time.

Finally, future developers of BUrl could develop some of the planned features that were cut due that could potentially improve the user experience of BUrl. One such feature was the 'Folder' system. While this feature currently exists in BUrl to give users a customized ability to sort their links in the application, the original idea was that each link would have a second order identifier so between users they could share link names (i.e. burl.live/folder/short_name). This feature was ultimately scrapped because of users not wanting to share link names in the first place. Furthermore, the elegance of just having the name following burl.live in a larger user base may be more valuable to have more permutations of link names. Another scrapped feature was groupings. This feature would have made it so a link on BUrl could be easily accessed and viewed for analysis by multiple users on the site outside of the creator if they were "grouped" in by them. After observing how BUrl was mainly used by individuals rather than teams, this feature was scrapped. However, if its user base shifts in the future, maintainers of the service should consider implementing this grouping feature.

References

1. “Bucknell University seal.” Wikipedia. https://en.wikipedia.org/wiki/Bucknell_University
2. “WCAG 2 Overview.” Web Accessibility Initiative (WAI), July 2005, www.w3.org/WAI/standards-guidelines/wcag/

Appendix

A1. Team Contributions

All BUrl team members contributed equally to the project. Individual strengths were leveraged and areas of growth and learning were pursued. The following is the general breakdown of primary contributions:

- **Alexa Horvath** - Created Figma mockups, branding, static pages (Landing, About), and accessibility audit. Assisted with some backend functionalities as well as testing. Also handled team planning and organizational tasks.
- **Ben Bonafede** - Assisted with the implementation of the backend functionality including the Google OAuth, Google Safe Browsing, and Open AI clients, along with the initial setup. Created the CI/CD pipeline within Gitlab to ensure security, testing, and deployment.
- **Brian Scotto** - Created user flow diagrams, assisted with settings page, privacy terms page, feedback feature, dark mode, and Testing.
- **Conor McNichols** - Assisted with backend functionality including analytics page development and styling.
- **Thao Nguyen** - Assisted with the Figma mockup, branding, and Landing page, made the analytics page for each link, and worked on testing.
- **Will Choi** - Created user flow diagrams, assisted with settings page, feedback feature, dark mode, and user testing.

Each team member wrote an equal portion of the project proposal, user manual, and tech report. All members were involved in the making of the BUrl video.